Complete validation programs

The whole declared validation programs. These samples and source-correction illustrations do not supply any uniform quantifier in the written proofs. Their cost is disclosed separately from the required finite premises.

`validation.g`

```
# Validation only: finite samples of uniform written proofs and
false-weakening controls.
# Required finite conclusions are separately in required_finite.g.
# Exact bounded projective-line models; uniform proofs are in the
manuscript.
Need258:=function(b,s) if not b then Error(s);fi;end;
Projectivity258:=function(M,p)
 local out,x,den,y;
 out:=[];
 for x in [0..p] do
 if x=p then
 if M[2][1] mod p=0 then y:=p;else y:=
 (M[1][1]*PowerModInt(M[2][1],-1,p)) mod p;fi;
 else den:=(M[2][1]*x+M[2][2]) mod p;
 if den=0 then y:=p;else y:=((M[1][1]*x+M[1][2])*PowerModInt(den,-1,p))
 mod p;fi;
 fi;Add(out,y+1);od;return PermList(out);
end;
String258:=function(rr)
 local i,j;
 if Length(Set(rr))<>Length(rr) or not ForAll(rr,x->Order(x)=2) then
 return false;fi;
 for i in [1..Length(rr)] do for j in [i+1..Length(rr)] do
 if j>i+1 and rr[i]*rr[j]<>rr[j]*rr[i] then return false;fi;
 od;od;return true;
end;
IP258:=function(G,rr)
 local ss,hs,i,j;
 ss:=Combinations([1..Length(rr)]);hs:=List(ss,s->Subgroup(G,rr{s}));
 for i in [1..Length(ss)] do for j in [1..Length(ss)] do
 if Intersection(hs[i],hs[j])<>hs[Position(ss,Intersection(ss[i],ss[j]))]
 then return false;fi;
 od;od;return true;
end;
RunValidation:=function()
 local p,rr,G,orders,a,b,c,C,src,changed,H,subs,inv,x,y,z,triple,D,cent,e
 xt,tgt,ms,dual,ds,i,j;
 for p in [13,37,61] do
 ms:=[[[0,1],[1,0]],[[1,0],[1,-1]],[[-1,0],[0,1]],[[1,-2],[2,-1]]];
 Need258(ForAll(ms,M->PowerModInt(DeterminantMat(M),(p-1)/2,p)=1),"consta
 nt determinant-square membership");
 rr:=List(ms,M->Projectivity258(M,p));
 G:=Group(rr);Need258(String258(rr),"constant string matrices");
 Need258(Size(G)=p*(p^2-1)/2 and IsSimpleGroup(G),"constant PSL image");
 Need258(Size(Subgroup(G,rr{[1..3]}))=Size(G) and
 Size(Subgroup(G,rr{[2..4]}))=Size(G),"two full ends");
 Need258(Order(rr[1]*rr[2])=3 and Order(rr[2]*rr[3])=p,"source3,p type");
 a:=First([1..p-1],x->x^2 mod
 p=p-1);C:=Projectivity258([[-a,0],[0,1]],p);
```

```
src:=List([[[0,1],[-1,0]],[[a,0],[1,-a]],[[-a,0],[0,a]]],M->Projectivity
258(M,p));
# GAP right-action permutations reverse composition of the displayed
column matrices.
Need258(List(src,s->s^C)=rr{[1..3]},"literal source basis-change
adapter");
Print("CONSTANT_FAMILY ",p," ORDER ",Size(G)," TYPE
",List([1..3],i->Order(rr[i]*rr[i+1])),"SOURCE_CONJUGACY_PASS\n");
od;
# All triples of each chosen maximal subgroup are searched without an IP
input.
for ds in [[41,60,5,3],[47,24,3,4],[73,24,3,4]] do
p:=ds[1];G:=PSL(2,p);
subs:=MaximalSubgroupClassReps(G);
if ds[2]=60 then tgt:=AlternatingGroup(5);else
tgt:=SymmetricGroup(4);fi;
H:=First(subs,x->Size(x)=ds[2] and IsomorphismGroups(x,tgt)<>fail);
Need258(H<>fail,"retained maximal subgroup exists in diagnostic field");
inv:=Filtered(Elements(H),x->Order(x)=2);triple:=fail;
for x in inv do for y in inv do for z in inv do
if Length(Set([x,y,z]))=3 and x*z=z*x and Order(x*y)=ds[3] and
Order(y*z)=ds[4] and Subgroup(H,[x,y,z])=H then
triple:=[x,y,z];break;fi;
od;if triple<>fail then break;fi;od;if triple<>fail then break;fi;od;
Need258(triple<>fail,"source local marking
exists");D:=Subgroup(H,triple{[1,2]});cent:=Centralizer(G,D);
Need258(Size(cent)=2 and Size(Centralizer(H,D))=1,"halfturn leaves
facet");
ext:=First(Elements(cent),x->Order(x)=2);rr:=Concatenation(triple,[ext])
;
Need258(String258(rr) and Subgroup(G,rr)=G and
Subgroup(G,rr{[2..4]})=G,"maximal facet extension");
Print("PROPER_END_FAMILY ",p," FACET ",Size(H)," FULL ",Size(G)," TYPE
",List([1..3],i->Order(rr[i]*rr[i+1])),"PASS\n");
od;
rr:=[(3,9)(4,10)(5,11)(6,12)(7,13)(8,14),(1,4)(2,14)(3,9)(5,13)(6,11)(7,
12),
(1,2)(3,7)(4,6)(8,14)(9,13)(10,12),(1,2)(4,14)(5,13)(6,12)(7,11)(8,10)];
G:=PSL(2,13);dual:=(1,3)(2,9)(4,7)(5,8)(6,12)(10,11)(13,14);
ext:=Concatenation(rr,[()]);
Need258(Length(Set(ext))=5 and Group(ext)=G and
ForAll(ext,x->x*x=()),"rankfive identity-marker countermodel");
for i in [1..5] do for j in [1..5] do if j>i+1 then
Need258(ext[i]*ext[j]=ext[j]*ext[i],"rankfive string
commutations");fi;od;od;
Need258(not String258(ext),"nonidentity condition is essential");
Print("RANK_BOUND_ALLOW_IDENTITY_FALSE_WEAKENING_RANK5_REJECTED\n");
# Omitting IP in an arbitrary group really enlarges the local class.
G:=DihedralGroup(IsPermGroup,12);inv:=Filtered(Elements(G),x->Order(x)=2
);rr:=fail;
for a in inv do for b in inv do for c in inv do
if Length(Set([a,b,c]))=3 and a*c=c*a and Subgroup(G,[a,b,c])=G
and Order(a*b)>2 and Order(b*c)>2 and not IP258(G,[a,b,c]) then
rr:=[a,b,c];break;fi;
od;if rr<>fail then break;fi;od;if rr<>fail then break;fi;od;
Need258(rr<>fail,"no blanket sggi implies C assertion");
Print("DIHEDRAL_NON_C_SGGI_CONTROL TYPE
",List([1,2],i->Order(rr[i]*rr[i+1])),"\n");
```

```
 # A centre-free group with the common-centralizer condition can have
 five
 # distinct involutory generators if the string relations are omitted.
 G:=AlternatingGroup(5);
 rr:=[(1,2)(3,4),(1,3)(2,4),(1,2)(3,5),(1,3)(2,5),(1,4)(2,5)];
 Need258(Length(Set(rr))=5 and Group(rr)=G,"no-string five-generator
 control");
 inv:=Filtered(Elements(G),x->Order(x)=2);
 for a in inv do for b in inv do if a*b<>b*a then
 Need258(Size(Centralizer(G,Subgroup(G,[a,b])))<=2,"pair-centralizer
 premise in control");
 fi;od;od;
 Need258(rr[1]*rr[3]<>rr[3]*rr[1],"actual omitted string relation");
 Print("RANK_BOUND_NO_STRING_FALSE_WEAKENING_A5_FIVE_MARKERS_REJECTED\n")
 ;
 Print("TOPIC_E_VALIDATION_PASS\n");
end;
RunValidation();QUIT;
```

pointed_models.g

```
# Validation of the uniform pointed-triple proof; not a finite proof
premise.
Need:=function(b,m) if not b then Error(m);fi;end;
ProjectivePerm:=function(M,F)
 local pts,n,im,v,w,pos;
 pts:=Elements(F);n:=Length(pts);im:=[];
 for pos in [1..n+1] do
  if pos<=n then v:=[pts[pos],One(F)];else v:=[One(F),Zero(F)];fi;
  w:=v*M;
  if IsZero(w[2]) then Add(im,n+1);else
  Add(im,Position(pts,w[1]/w[2]));fi;
 od;return PermList(im);
end;
CheckModel:=function(prm)
 local q,k,F,o,z,t,a,b,c,r,pp,G,endA,endB,mid,p,d,sq,target,invfields,e,i
 mg,aut;
 q:=prm[1];k:=prm[2];F:=GF(q);o:=One(F);z:=Zero(F);t:=Z(q)^((q-1)/k);
 a:=[[-t^-1,o],[z,o]];b:=[[t,z],[z,o]];c:=[[o,z],[o+t,-t]];r:=[[-o,o],[z,
 o]];
 pp:=List([a,b,c],M->ProjectivePerm(M,F));G:=Group(pp);endA:=Group(pp{[1,
 2]});endB:=Group(pp{[2,3]});mid:=Group(pp[2]);
 Need(Order(pp[1]*pp[2])=2 and Order(pp[2]*pp[3])=2 and
 Order(Product(pp))=2,"relations");
 Need(Intersection(endA,endB)=mid and
 Size(Intersection(Group(pp[1]),mid))=1 and
 Size(Intersection(Group(pp[3]),mid))=1,"intersection");
 p:=Characteristic(F);d:=DegreeOverPrimeField(F);sq:=p=2 or
 t^((q-1)/2)=o;
 if sq and p<>2 then target:=q*(q^2-1)/2;else target:=q*(q^2-1);fi;
 Need(Size(G)=target and Size(endA)=q*k and Size(endB)=q*k,"orders");
 invfields:=Filtered([1..d-1],e->t^(p^e)=t^-1);
 if Length(invfields)>0 then
  e:=invfields[1];img:=List([a,b,c],M->ProjectivePerm(r*List(M,row->List(
  row,x->x^(p^e)))*r,F));
  Need(img=[pp[1]^-1,pp[1]^2*pp[2],pp[3]],"semilinear images");
  aut:=GroupHomomorphismByImages(G,G,pp,img);Need(aut<>fail and
  IsBijective(aut),"actual reversing automorphism");
```

```
 fi;
 Print("POINTED
 ",rec(q:=q,k:=k,group_order:=Size(G),end_order:=Size(endA),intersection:
 =Size(Intersection(endA,endB)),field_reversal:=invfields),"\n");
end;
for prm in
[[4,3],[5,4],[8,7],[9,4],[13,6],[16,15],[25,6],[25,12],[27,26],[49,24]]
do CheckModel(prm); od;
Print("POINTED_PASS\n");QUIT;
```

source_clause_571.g

```
# Validation of the explicit type (3,5,3) correction at p=571; not a
finite proof premise.
Need:=function(b,m) if not b then Error(m);fi;end;
ProjectivePerm:=function(M,F)
 local pts,n,im,v,w,pos;
 pts:=Elements(F);n:=Length(pts);im:=[];
 for pos in [1..n+1] do
  if pos<=n then v:=[pts[pos],One(F)];else v:=[One(F),Zero(F)];fi;
  w:=v*M;
  if IsZero(w[2]) then Add(im,n+1);else
  Add(im,Position(pts,w[1]/w[2]));fi;
 od;return PermList(im);
end;

Check571:=function()
 local p,F,o,z,root,bval,w,y,x,A,B,C,J,r,ss,sols,pr,ends,whole,tr,disc,sq
 ,matched,mat;
 p:=571;F:=GF(p);o:=One(F);z:=Zero(F);J:=[[z,o],[-o,z]];matched:=0;
 for root in Filtered(Elements(F),v->v^2=5*o) do
  bval:=(-o+root)/2;
  sols:=[];
  for w in Elements(F) do
   y:=First(Elements(F),v->v^2+bval*v+w^2-w+o=z);
   if y<>fail and 2*y+bval<>z then Add(sols,[w,y]);break;fi;
  od;
  Need(Length(sols)=1,"A5 seed");w:=sols[1][1];y:=sols[1][2];x:=y+bval;
  A:=[[w,x],[y,o-w]];B:=A^-1*J;
  for ss in Elements(F) do
   r:=(y-(o-2*w)*ss)/(2*y+bval);C:=[[r,ss],[ss,o-r]];
   if DeterminantMat(C)<>o then continue;fi;
   pr:=List([A,B,C],M->ProjectivePerm(M,F));ends:=[Group(pr{[1,2]}),Group
   (pr{[2,3]})];whole:=Group(pr);
   Need(List(pr,Order)=[3,5,3],"type");Need(Order(pr[1]*pr[2])=2 and
   Order(pr[2]*pr[3])=2 and Order(Product(pr))=2,"rotation relations");
   Need(List(ends,Size)=[60,60] and
   Size(Intersection(ends[1],ends[2]))=5,"A5 ends and exact IP");
   Need(Size(whole)=p*(p^2-1)/2,"whole PSL571 order");
   tr:=TraceMat(A*C)-TraceMat(A)*TraceMat(C)/2;Need(tr<>z,"nonzero
   reversal obstruction");
   Print("PSL571 ",rec(matrices:=List([A,B,C],M->List(M,row->List(row,Int
   ))),order:=Size(whole),local_orders:=List(ends,Size),intersection:=Siz
   e(Intersection(ends[1],ends[2])),trace_obstruction:=Int(tr)),"\n");mat
   ched:=matched+1;
  od;
 od;
 Need(matched=2,"two [3,5,3] extensions");Print("PSL571_PASS\n");
```

```
end;
Check571();QUIT;
```

psl2_square_witnesses.g

```
# Exact primitive-element witnesses for the six small cases left by the
# proposed character-sum bound. No construction of the large compositum.
LogTo("research/leemans/psl2_square_witnesses.log");

SquareWitness := function(p,a,b)
  local A,B,j1,j2base,u,x,P,i,ui,coeff,Q,n,k,j2,val,root,
        primitiveCount,squareCount,firstK,firstVal,firstJ2,c,normpoly,
        candidateValues;
  A:=GF(p^a); B:=GF(p^b);
  j1:=Z(p^a); j2base:=Z(p^b);
  u:=j1+j1^-1;
  x:=Indeterminate(A);
  P:=One(A)+0*x;
  for i in [0..a-1] do
    ui:=u^(p^i);
    P:=P*((ui^2-4)*(x^2+1)^2-4*ui^2*x^2);
  od;
  coeff:=CoefficientsOfUnivariatePolynomial(P);
  if not ForAll(coeff,c->c^p=c) then Error("Norm coefficients not in
  prime field"); fi;
  coeff:=List(coeff,IntFFE);
  Q:=p^b; n:=Q-1;
  primitiveCount:=0; squareCount:=0; firstK:=fail;
  for k in [1..n-1] do
    if Gcd(k,n)<>1 then continue; fi;
    primitiveCount:=primitiveCount+1;
    j2:=j2base^k;
    val:=Zero(B);
    for c in Reversed(coeff) do val:=val*j2+c*One(B); od;
    if val<>Zero(B) and val^(n/2)=One(B) then
      squareCount:=squareCount+1;
      if firstK=fail then firstK:=k; firstVal:=val; firstJ2:=j2; fi;
    fi;
  od;
  if firstK=fail then Error("No witness"); fi;
  if Order(j1)<>p^a-1 or Order(firstJ2)<>p^b-1 then Error("Not
  primitive"); fi;
  Print("CASE p=",p," a=",a," b=",b,"\n");
  Print("j1=Z(",p,"^",a,"), j2=Z(",p,"^",b,")^",firstK,"\n");
  Print("minimal polynomial j1 coefficients(low-to-high)=",
    List(CoefficientsOfUnivariatePolynomial(MinimalPolynomial(GF(p),j1)),
    IntFFE),"\n");
  Print("minimal polynomial j2 coefficients(low-to-high)=",
    List(CoefficientsOfUnivariatePolynomial(MinimalPolynomial(GF(p),first
    J2)),IntFFE),"\n");
  Print("norm polynomial coefficients(low-to-high)=",coeff,"\n");
  Print("P(j2)=Z(",p,"^",b,")^",LogFFE(firstVal,j2base),
        "; quadratic character=",firstVal^(n/2),"\n");
  Print("fixed-j1 primitive j2 count=",primitiveCount,
        "; successful count=",squareCount,
        "; inverse-pair trace
        counts=",primitiveCount/2,",",squareCount/2,"\n");
  Print("PASS\n");
```

```
end;

Print("GAP ",GAPInfo.Version,"\n");
for triple in [[3,3,5],[5,3,5],[3,3,7],[3,5,7],[3,5,9],[3,7,9]] do
  CallFuncList(SquareWitness,triple);
od;
LogTo();
QUIT;
```

psl2_nested_witnesses.g

```
# Exact same-field witnesses for the small nested-degree cases of Conj.
6.1.
LogTo("research/leemans/psl2_nested_witnesses.log");
NestedWitness:=function(p,e)
  local K,j1,j2,u,v,s,q,k,value,omega,firstK,firstValue,firstJ2,
        primitiveCount,squareCount,zeroCount;
  K:=GF(p^e); q:=p^e;
  j1:=Z(q); u:=j1+j1^-1; s:=u^2;
  firstK:=fail; primitiveCount:=0; squareCount:=0; zeroCount:=0;
  for k in [1..q-2] do
    if Gcd(k,q-1)<>1 then continue; fi;
    primitiveCount:=primitiveCount+1;
    j2:=Z(q)^k; v:=j2+j2^-1;
    value:=(s-4)*(j2^2+1)^2-4*s*j2^2;
    omega:=s*v^2-4*(s+v^2);
    if value<>j2^2*omega then Error("Quartic normalization fails"); fi;
    if value=Zero(K) then
      zeroCount:=zeroCount+1;
    elif value^((q-1)/2)=One(K) then
      squareCount:=squareCount+1;
      if firstK=fail then firstK:=k; firstValue:=value; firstJ2:=j2; fi;
    fi;
  od;
  if firstK=fail then Error("No nonzero square witness"); fi;
  if Order(j1)<>q-1 or Order(firstJ2)<>q-1 then Error("Not primitive");
  fi;
  Print("CASE p=",p," e1=e2=",e,"\n");
  Print("j1=Z(",p,"^",e,"), j2=Z(",p,"^",e,")^",firstK,"\n");
  Print("minimal polynomial j1 coefficients(low-to-high)=",
    List(CoefficientsOfUnivariatePolynomial(MinimalPolynomial(GF(p),j1)),
    IntFFE),"\n");
  Print("minimal polynomial j2 coefficients(low-to-high)=",
    List(CoefficientsOfUnivariatePolynomial(MinimalPolynomial(GF(p),first
    J2)),IntFFE),"\n");
  Print("quartic(j2)=Z(",p,"^",e,")^",LogFFE(firstValue,Z(q)),
    "; quadratic character=",firstValue^((q-1)/2),"\n");
  Print("fixed-j1 primitive j2 count=",primitiveCount,
    "; nonzero-square count=",squareCount,"; zero count=",zeroCount,
    "; inverse-pair trace
    counts=",primitiveCount/2,",",squareCount/2,"\n");
  Print("PASS\n");
end;
Print("GAP ",GAPInfo.Version,"\n");
for pair in [[3,3],[3,5],[5,3],[7,3],[11,3]] do
CallFuncList(NestedWitness,pair); od;
LogTo();
QUIT;
```

psl2_intersection_check.g

```
# Exact matrix check of Conjecture 6.2 at its first residual case q=3^15.
# Enumerates the SMALL containing SL(2,27), not PSL(2,3^15).
LogTo("research/leemans/psl2_intersection_check.log");
p:=3;; e1:=3;; e2:=5;; q:=p^(e1*e2);;
L:=GF(q);; one:=One(L);;
j1:=Z(p^e1)*one;; j2base:=Z(p^e2)*one;;
u:=j1+j1^-1;;
for k in [1..p^e2-2] do
  if Gcd(k,p^e2-1)<>1 then continue; fi;
  j2:=j2base^k;; v:=j2+j2^-1;;
  omega:=u^2*v^2-4*(u^2+v^2);;
  if omega<>Zero(L) and omega^((q-1)/2)=one then break; fi;
od;
delta:=omega^((q+1)/4);;
if delta^2<>omega then Error("No square root found"); fi;
alpha:=[[u,u+2],[u-2,u]]/2;;
beta:=[[2+u,u],[-u,2-u]]/2;;
gamma:=[[v*(u-2),delta],[delta,v*(u+2)]]/(2*u);;
T:=[[one,2/u],[-one,Zero(L)]];;
Unip:=t->[[one,t],[Zero(L),one]];;
Amat:=w->[[one,(2-w)/w],[-w,w-one]];;
t:=delta/(2*u*v)+1/v-1/2*one-1/u;;
if beta^T<>Unip(one) or alpha^T<>Amat(u) or (gamma^T)^Unip(t)<>Amat(v)
then
  Error("Normalization identity fails");
fi;
small:=SL(2,p^e1);;
intersection:=[];;
for M in Elements(small) do
  Mlarge:=List(M,row->List(row,x->x*one));;
  N:=Mlarge^Unip(t);;
  if ForAll(Flat(N),x->x^(p^e2)=x) then Add(intersection,Mlarge); fi;
od;
if Length(intersection)<>2*p then Error("Unexpected containing
intersection"); fi;
if not ForAll(intersection,M->M[2][1]=Zero(L) and M[1][1]=M[2][2] and
    M[1][1]^2=one and M[1][2]^p=M[1][2]) then Error("Intersection shape
    fails"); fi;
Print("GAP ",GAPInfo.Version," p=",p," e1=",e1," e2=",e2," q=",q,"\n");
Print("primitive exponents 1,",k,"; normalization identities
verified\n");
Print("SL(2,27) enumerated=",Size(small)," intersection
size=",Length(intersection),
      " projective intersection size=",Length(intersection)/2,"\n");
Print("PASS: containing intersection is projective U(F_3)=<beta>\n");
LogTo();
QUIT;
```